Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА ИУ7 ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ

ОТЧЕТ ПО УЧЕБНОЙ ПРАКТИКЕ
(Проектно-технологическая практика)

Студент	**Дудырев Д С**
Группа	**ИУ7-12Б**
Тип практики	**Практикум**
Название предприятия	**НУК ИУ МГТУ им. Н.Э. Баумана**

Студент	_____ *подпись, дата*	**Дудырев Д С** *фамилия, и.о.*
Руководитель практики	_____ *подпись, дата*	**Кострицкий А. С.** *фамилия, и.о.*
Руководитель практики	_____ *подпись, дата*	**Ломовской И. В.** *фамилия, и.о.*

Оценка _____

2024 г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ 1

ВВЕДЕНИЕ 2

Сценарии командной оболочки 4

Сравнение целых чисел 4

Тесты 4

Код скрипта 4

Сравнение подстрок 6

Тесты 6

Код скрипта 6

Сравнение ЧПТ (не экспоненциальной формы) 8

Тесты 8

Код скрипта 9

Сравнение ЧПТ (любой формы записи) 11

Тесты 11

Код скрипта 11

ЗАКЛЮЧЕНИЕ 13

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 1

ВВЕДЕНИЕ

Написать скрипты для командной оболочки, которое производят сравнение двух текстовых файлов по следующим правилам:

1. Сравнение последовательности целых чисел, отделенных пробелами, в файлах. Производиться анализ только последовательности чисел, игнорирую другие символы
2. Строгое равнение текста файлов после вхождения подстроки «string:». Строгое сравнение подразумевает сравнение с учётом разницы в пробельных символах и символах окончания строки.
3. Сравнение последовательности чисел с плавающей с точкой (ЧПТ), записанных не в экспоненциальной форме. Производиться анализ только последовательности чисел, игнорирую другие символы. Сравнение чисел происходит, как сравнение текста
4. Сравнение последовательности чисел с плавающей с точкой (ЧПТ), записанных не в экспоненциальной форме и в экспоненциальной форме. Производиться анализ только последовательности чисел, игнорирую другие символы. Сравнение чисел происходит, как сравнение текста

Написать скрипты для тестирования реализованных сценариев командной оболочки и подготовить файлы для проведения тестов.

Задачи:

1. Написать скрип для сравнения последовательной целых чисел
   1.1. Реализовать функцию выделения из текстового файла целых чисел, отделенных от других символов пробелами
   1.2. Реализовать проверку существование переданных файлов
   1.3. Реализовать чтение переданных файлов
   1.4. Реализовать сравнение выделенных числовых последовательностей
   1.5. Реализовать работу параметра -v (verbose)
2. Выделение из текста дробных чисел в не экспоненциальной форме

2.1. Реализовать функцию выделения из текстового файла дробных чисел, не учитывая экспоненциальную форму
2.2. Реализовать проверку существование переданных файлов
2.3. Реализовать чтение переданных файлов
2.4. Реализовать сравнение выделенных числовых последовательностей
2.5. Реализовать работу параметра -v (verbose)

3. Выделение из текста дробных чисел в экспоненциальной форме
   3.1. Реализовать функцию выделения из текстового файла дробных чисел, записанных в любой форме
   3.2. Реализовать проверку существование переданных файлов
   3.3. Реализовать чтение переданных файлов
   3.4. Реализовать сравнение выделенных числовых последовательностей
   3.5. Реализовать работу параметра -v (verbose)
4. Выделение из текста строки после подстроки “string:”
   4.1. Реализовать функцию выделения фрагмента текста после подстроки “string:”
   4.2. Реализовать проверку существование переданных файлов
   4.3. Реализовать чтение переданных файлов
   4.4. Реализовать сравнение выделенных фрагментов
   4.5. Реализовать работу параметра -v (verbose)
5. Составить тестовые файлы для проверки корректности работы скрипта
6. Написать скрипт test.sh для автоматизации проверки корректности работы скрипта

Сценарии командной оболочки

Сравнение целых чисел

Тесты

Номер теста	Описание	Ожидаемый результат
1	Простые последовательности (1 2 3 4 5) одинаковые в обоих файлах	0
2	Последовательности сложных чисел: отрицательные, начинаются с нулей	0
3	file_1 содержит просто числа, file_2 содержит ту же последовательность, но между числами стоят слова	0
4	Две разные последовательности	1
5	Один пустой файл	1
6	Два пустых файла	0
7	Два файла не содержащих числа	0
8	один из указанных файлов не существует	2

Код скрипта

Объявление функции для выделения из текста целых чисел, отделенных от другого содержимого файла пробелами. Код функции проходит по переданной строке при помощи цикла for и записывает, если элемент соответствует маске целого числа

```
function extruct_numbers() {
  numbers=""
  for num in $(cat $1); do
    if [[ $num =~ ^-?[0-9]+$ ]]; then
  echo "$num "
  fi
  done
}
```

Проверка был ли передан параметр работы скрипты

```
if [[ "$1" =~ -[a-zA-z] ]]; then
  parameters=$1
  shift
fi
```

Проверка корректности переданных файлов. Если один из аргументов не является файлом или не существует, то скрипт завершиться с кодом ошибки 2

```
for file in $@; do
  if [[ ! -f $file ]]; then
    if [[ "$parameters" = "-v" ]]; then
      echo $file will not find or is not a file
    fi
    exit 2
  fi
done
```

Вывод содержимого переданных файлов, если был передан параметр v

```
if [[ "$parameters" = "-v" ]]; then
 echo $1 contain: $(cat $1)
 echo $2 contain: $(cat $2)
fi
```

Присваивание переменным numbers_1 и numbers_2 выделенных последовательности чисел из файлов

```
numbers_1=$(extruct_numbers $1)
numbers_2=$(extruct_numbers $2)
```

Вывод содержимого перменных numbers_1 и numbers_2, если был передан параметр v

```
if [[ "$parameters" = "-v" ]]; then
 echo $1 contain numbers: $numbers_1
 echo $2 contain numbers: $numbers_2
fi
```

Сравнение значений переменных numbers_1 и numbers_2

```
if [ "$numbers_1" == "$numbers_2" ]; then
  if [[ "$parameters" = "-v" ]]; then
    echo result compare: Matched
  fi
  exit 0
else
  if [[ "$parameters" = "-v" ]]; then
    echo result compare: Different
  fi
  exit 1
fi
```

Сравнение подстрок

Тесты

Номер теста	Описание	Ожидаемый результат
1	Две идентичных строки содержащих одно вхождение «string:»	0
2	Две идентичных строки содержащих несколько вхождений «string:»	0
3	Один файл содержит «string:», а другой нет	1
4	Два одинаковых файла содержащих «string:», несколько строк	0
5	до «string:» файлы немного отличаются, после идентичны	0
6	Два файла с одинаковым содержанием, но в одном всё расположено в одну строку, в другом разбито на несколько	1
7	Один из файлов не существует	2

Код скрипта

Объявлена функция для выделения из файла фрагмента текста после подстроки «string:». В коде функции сепаратор для цикла for устанавливается на символ

переноса строки. В каждой строке файла ищется вхождение подстроки «string:», если данная подстрока найдена, то всё что было до неё обрезается, оставщаяся часть строки добавляется возвращаемому значению, флаг принимает значение при котором все последующие строки добавляются к возвращаемому значению

```
function extract() {
   string=""
   flag=1
   OLD_IFS=$IFS
   IFS="\\"
   for substring in $(cat $1); do
     if [[ "$flag" = "0" ]]; then
       echo "$substring "
     elif [[ "$substring" =~ .*string: ]]; then
       echo "$( echo $substring | sed "s/.*string://" )"
       flag=0
     fi
   done
   IFS=$OLD_IFS
}
```

Проверка был ли передан параметр работы скрипты

```
if [[ "$1" =~ -[[:alpha:]] ]; then
  parameters=$1
  shift
fi
```

Проверка корректности переданных файлов. Если один из аргументов не является файлом или не существует, то скрипт завершиться с кодом ошибки 2

```
for file in $@; do
  if [[ ! -f $file ]]; then
    if [[ "$parameters" = "-v" ]]; then
      echo $file will not find or is not a file
    fi
    exit 2
  fi
done
```

Вывод содержимого переданных файлов, если был передан параметр v

```
if [[ "$parameters" = "-v" ]]; then
 echo $1 contain: $(cat $1)
 echo $2 contain: $(cat $2)
fi
```

Присваивание переменным substring_1 и substring_2 выделенных последовательности чисел из файлов

```
substring_1=$(extract $1)
substring_2=$(extract $2)
```

Вывод содержимого перменных substring_1 и substring_2, если был передан параметр v

```
if [[ "$parameters" = "-v" ]]; then
 echo $1 contain numbers: $substring_1
 echo $2 contain numbers: $substring_2
fi
```

Сравнение значений переменных substring_1 и substring _2

```
if [ "$substring_1" == "$substring_2" ]; then
  if [[ "$parameters" = "-v" ]]; then
    echo result compare: Matched
  fi
  exit 0
else
  if [[ "$parameters" = "-v" ]]; then
    echo result compare: Different
  fi
  exit 1
fi
```

Сравнение ЧПТ (не экспоненциальной формы)

Тесты

Номер теста	Описание	Ожидаемый результат
1	Два одинаковых файла	0
2	Два файла, в которых сами числа одинаковы, но расположены в разных местах	1

3	Разные числа в файлах	1
4	одинаковые числа, но запись разная например, 1 и 001, 2.00 и 002	1
5	один из файлов не существует	2
6	В file_1.txt содержится число в экспоненциальной форме, в остальном наборы чисел идентичны	0

Код скрипта

Объявление функция для выделения из текста дробных чисел, записанных не в экспоненциальной форме. Функция с помощью цикла for проходит по переданной строке и записывает элемент, если он соответствует маске, обрезая текст, который идет до или после числа.

```
function extruct_numbers() {
   numbers=""
   for num in $(cat $1); do
     if [[ $num =~ \-[0-9]+(\.[0-9]+)? && ! $num =~ [0-9]+e[\-+]?[0-9]+ ]]; then
       echo $num | sed -E 's/[^0-9]*([0-9]+(\.[0-9]+)?)[^0-9]*/-\1/'
     elif [[ $num =~ [0-9]+(\.[0-9]+)? && ! $num =~ [0-9]+e\[\-+]?[0-9]+ ]]; then
       echo $num | sed -E 's/[^0-9]*([0-9]+(\.[0-9]+)?)[^0-9]*/\1/'
     fi
   done
}
```

Проверка был ли передан параметр работы скрипты

```
if [[ "$1" =~ -[a-zA-z] ]]; then
  parameters=$1
  shift
fi
```

Проверка корректности переданных файлов. Если один из аргументов не является файлом или не существует, то скрипт завершиться с кодом ошибки 2

```
for file in $@; do
  if [[ ! -f $file ]]; then
    if [[ "$parameters" = "-v" ]]; then
      echo $file will not find or is not a file
    fi
    exit 2
  fi
done
```

Вывод содержимого переданных файлов, если был передан параметр v

```
if [[ "$parameters" = "-v" ]]; then
 echo $1 contain: $(cat $1)
 echo $2 contain: $(cat $2)
fi
```

Присваивание переменным numbers_1 и numbers_2 выделенных последовательности чисел из файлов

```
numbers_1=$(extruct_numbers $1)
numbers_2=$(extruct_numbers $2)
```

Вывод содержимого перменных numbers_1 и numbers_2, если был передан параметр v

```
if [[ "$parameters" = "-v" ]]; then
 echo $1 contain numbers: $numbers_1
 echo $2 contain numbers: $numbers_2
fi
```

Сравнение значений переменных numbers_1 и numbers_2

```
if [ "$numbers_1" == "$numbers_2" ]; then
  if [[ "$parameters" = "-v" ]]; then
    echo result compare: Matched
  fi
  exit 0
else
  if [[ "$parameters" = "-v" ]]; then
    echo result compare: Different
  fi
  exit 1
fi
```

Сравнение ЧПТ (любой формы записи)

Тесты

Номер теста	Описание	Ожидаемый результат
1	Два одинаковых файла	0
2	Два одинаковых файла, но содержится текст	0
3	Два фалйа, где file_1 содержит все числа, file_2 содержит часть file_1	1
4	Одинаковые числа разная запись	1
5	Один файл не существует	2

Код скрипта

Была объявлена функция для выделения из текста дробных чисел, записанных в любой форме. Функция с помощью цикла for проходит по переданной строке и записывает элемент, если он соответствует маске, обрезая текст, который идет до или после числа.

```
function extruct_numbers() {
   numbers=""
   for num in $(cat $1); do
     if [[ $num =~ \-[0-9]+(\.[0-9]+(e[+-]?[0-9])?)? ]]; then
       echo $num | sed -E 's/[^0-9]*([0-9]+(\.[0-9]+(e[+-?[0-9])?)?)[^0-9]*/-\1/'
     elif [[ $num =~ [0-9]+(\.[0-9]+(e[+-]?[0-9])?)? ]]; then
       echo $num | sed -E 's/[^0-9]*([0-9]+(\.[0-9]+(e[+-]?[0-9])?)?)[^0-9]*/\1/'
     fi
   done
}
```

Проверка был ли передан параметр работы скрипты

```
if [[ "$1" =~ -[a-zA-z] ]]; then
  parameters=$1
  shift
fi
```

Проверка корректности переданных файлов. Если один из аргументов не является файлом или не существует, то скрипт завершиться с кодом ошибки 2

```
for file in $@; do
  if [[ ! -f $file ]]; then
    if [[ "$parameters" = "-v" ]]; then
      echo $file will not find or is not a file
    fi
    exit 2
  fi
done
```

Вывод содержимого переданных файлов, если был передан параметр v

```
if [[ "$parameters" = "-v" ]]; then
 echo $1 contain: $(cat $1)
 echo $2 contain: $(cat $2)
fi
```

Присваивание переменным numbers_1 и numbers_2 выделенных последовательности чисел из файлов

```
numbers_1=$(extruct_numbers $1)
numbers_2=$(extruct_numbers $2)
```

Вывод содержимого перменных numbers_1 и numbers_2, если был передан параметр v

```
if [[ "$parameters" = "-v" ]]; then
 echo $1 contain numbers: $numbers_1
 echo $2 contain numbers: $numbers_2
fi
```

Сравнение значений переменных numbers_1 и numbers_2

```
if [ "$numbers_1" == "$numbers_2" ]; then
  if [[ "$parameters" = "-v" ]]; then
    echo result compare: Matched
  fi
  exit 0
else
  if [[ "$parameters" = "-v" ]]; then
    echo result compare: Different
  fi
  exit 1
fi
```

ЗАКЛЮЧЕНИЕ

Были выполнены следующие задачи:

1. Написаны скрипты для сравнения
   1.1. Последовательности целых чисел
   1.2. Последовательности дробных чисел в не экспоненциальной форме
   1.3. Последовательности дробных чисел в экспоненциальной форме
   1.4. Сравнения фрагмента текста после вхождения определенной подстроки
2. Сформированы текстовые файлы для проверки корректной работы скриптов
3. Написаны скрипты для автоматизации проверки корректной работы скриптов сравнения

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

– Курс: Проектно-технологическая практика: сайт. – URL: https://e-learning.bmstu.ru/iu7/course/view.php?id=76

– Блум Р., Бреснахэн К. Командная строка Linux и сценарии оболочки. Библия пользователя